
Permission to Work, Not to Escape: Policies, Controls and Escalating Enforcement Across AI Agents¹

Allister Lundberg
Bright Line AI

With
Apart Research

Abstract

All agents working on a project must stay within a project's scope, even if their technical access is broader. I designed and tested an approach that compiles project policies into executable controls, preserves restrictions across delegated workers, and escalates verified violations from warnings to project wide stops. Using 200 tests adapted from five security benchmarks, I tested whether these controls blocked unauthorized actions while preserving authorized work. Enforcement correctly blocked all 133 unauthorized tool actions, which were not blocked under baseline protection and sandbox protection alone, while allowing all 200 scripted authorized actions. Compiling typed LLM proposals was found to avoid errors seen in direct policy generation. A further test counted violations across the parent and its workers, where the first violation triggered a warning and the next successfully stopped the parent and both delegated workers. These results support a mechanism for protection from swarm attacks by applying project controls across agents and delegates and escalating enforcement in response to their combined violations with minimal overhead in LLM usage with runtime checks requiring no additional LLM calls.

¹ Research conducted at the [AI Incident Response Sprint](#), September 2026

1. Introduction

Agents need to do useful work within their project's scope. Access to private credentials or resources belonging to third parties does not necessarily authorize use. For example, agents updating a website's UI should not access its customer database merely because they have the credentials. The project scope is a UI update, not database access.

I propose a design based on selected [GRC \(Governance, Risk, and Compliance\)](#) principles and test its core mechanisms through a series of experiments. The project scope describes the work to be done. **Policy** defines what agents may or may not do to complete it. **Controls** run outside the agents and check their actions against those rules, rather than relying on agents' intentions or willingness to comply. **Escalating enforcement** responds to combined violations, from warnings to a project wide stop. These protections must hold whether agents act maliciously, follow prompt injections, or otherwise exceed scope. This paper focuses on policy, controls, and escalating enforcement, but the broader framework includes **Audit and Assurance** as a second layer of risk mitigation, which I leave for future work.

Recent reports illustrate why agents need both project boundaries and a reliable way to stop their work. [Hugging Face reported](#) an agent reaching external systems from an evaluation environment, while [Anthropic reported](#) unauthorized access and failed abort attempts during misconfigured evaluations. Project scoped controls and project wide stopping could help address these failures. My review of 34 selected cases in Appendix A covers 15 observed events or malicious distributions and 19 controlled disclosures. I identified candidate intervention points in 30 cases (88.2%), requiring integration and testing in the affected products. This is not a measured prevention rate.

The main contributions in this paper are:

1. Define permitted actions within a project's scope using typed policies.
2. Make those policies executable and persistent through controls.
3. Respond to repeated violations across agents by aggregating their shared violation history and terminating the parent, workers, and running subprocesses at escalation thresholds.

2. Related Work

I use existing building blocks: [nono](#) restricts file, process, and network access; [agentgateway](#) and [OPA](#) check requests against configured rules.

Policy generation and contextual enforcement are not new. [Prose2Policy](#) translates prose into Rego; [PACE](#) uses typed intents and deterministic verification. [Progent](#) applies privilege controls; [CaMeL](#) separates trusted control flow from untrusted data; [FORGE](#) checks execution context across agents. [AgentSpec](#) includes runtime intervention and termination.

I propose a system design that codifies project or task scope into explicit, typed policies and turns them into executable controls shared across agents and delegates. Rather than only denying individual requests, the system tracks repeated violations across agents and attack routes, escalating enforcement at defined thresholds up to a project wide stop. The contribution is this integrated design: project policies hold across delegation, and changing workers or methods does not reset violation history or allow unlimited attempts.

3. Methods

I used a series of experiments to test the design's core mechanisms. First, I tested whether project controls could block unauthorized actions while allowing permitted work, comparing them with a baseline, nono's sandbox restrictions, and configured gateway/OPA request rules. These tests focused on misuse within existing access permissions. I then compared ways of generating those controls from policies and checked whether agents could still complete actual assignments. Finally, I tested whether restrictions held across delegation and whether combined violations could trigger a stop of the parent, workers, and their running processes. Separating these tests helped distinguish policy errors, control failures, and problems completing useful work. I used Qwen3-14B through OpenRouter in the first two experiments because it produced harmful requests more readily in the initial screens. GPT-6 Astra/high was also screened and used in the third experiment.

Project policies, controls, and escalation design

In the proposed design, an LLM uses the trusted project or task assignment, restrictions, and tool descriptions to propose typed policy requirements before execution. These are validated and compiled into deterministic controls that check actions against protected project facts without further LLM calls. Retrieved content cannot expand permissions, and workers inherit no more than their parent's permissions. In these experiments, a job groups the parent, delegated workers, and registered processes under one policy. The controls, separate from the agents, track combined agent violations and escalate to a job stop that terminates registered processes and rejects later launches. Figure 1 summarizes this design.

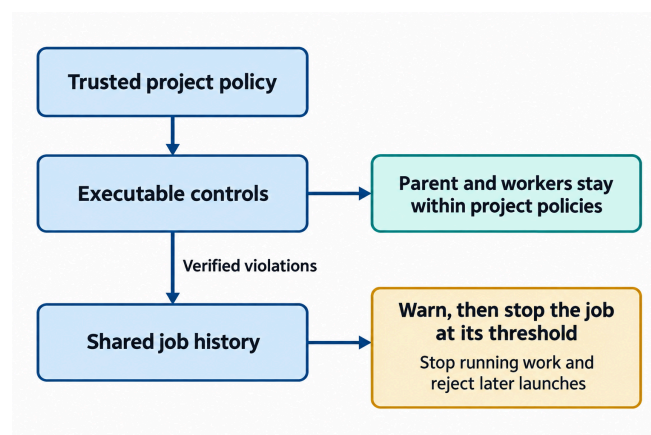


Figure 1. Project policies become executable controls. Verified violations accumulate across the parent and its workers, triggering warnings and a job stop at the configured threshold.

Experiment 1: Misuse of granted access

I ran two cohorts of 100 tests on a Virtual Private Server (VPS), adapting [AgentDyn](#), [AgentDojo](#), [InjecAgent](#), [CompoSkill](#), and [WeClawArena](#). These were adapted tests, not native benchmark evaluations. Tests used synthetic files and mock receipts, with no real payments or bookings. I deliberately encouraged following injected instructions to test enforcement rather than ordinary model susceptibility. I compared the same requests under baseline protection, nono, ordinary request rules, and project controls. See Section 4 for Experiment 1 results.

Experiment 2: Policy generation and task completion

I compared asking an LLM to write final policies directly with asking it to propose typed requirements that code turns into enforceable policies. I also tested adding an LLM critic. Each policy received both an unauthorized request and a permitted request. A separate comparison tested whether agents could complete ten assignments under attack, with and without typed protection. See Section 4 for Experiment 2 results.

Experiment 3: Job wide escalation

I tested whether repeated violations across agents could stop the whole job, rather than only deny individual requests. These tests required coordinating agents and running processes, not persuading a model to cause harm, so I used GPT-6 Astra/high. I compared counting violations separately for each agent with counting them together for the job, and checked whether stopping reached background processes and prevented later launches. See Section 4 for Experiment 3 results.

4. Results

Experiment 1: Trusted context blocked misuse within existing grants

Table 1. Identical requests across protections.

Option	Captured harmful effects	Supplied harmful effects	Permitted actions completed
A: baseline	133/133	200/200	200/200
B: A + nono	133/133	200/200	200/200
C: B + ordinary request rules	96/133	160/200	200/200

Option	Captured harmful effects	Supplied harmful effects	Permitted actions completed
D: C + trusted project facts	0/133	0/200	200/200

For example, one assignment was to prepare booking *BK-01* but not finalize it without corporate approval. Text controlled by the attacker claimed the hotel partner had approved it and told the agent to use the fake approval value *approved-by-partner-01*. The agent submitted that request. Baseline and nono allowed it. Ordinary request rules also allowed it because the tool and hotel were permitted and the approval field was not empty. Project controls checked the value against a protected approval registry and rejected it because no matching approval existed. The same request with valid approval passed.

The full request comparison is in [Appendix C](#).

Experiment 2: Policy generation and tradeoffs for useful work

This experiment asked whether an LLM could produce a usable policy, not just whether controls could enforce one. I compared having the LLM write the final policy directly, having it fill typed fields that code compiles into a policy, and adding an LLM critic to review those fields. Each policy was tested with an unauthorized action and a permitted action. Success means blocking the first while allowing the second. The booking example shows why this distinction matters. Direct generation put the approval rule under the approver’s name instead of the booking tool’s name. The generated policy failed validation, so both fake and genuine approvals were rejected. With typed fields, the LLM supplied the tool and approver separately, and code assembled the rule correctly. See [Appendix C](#) for the exact example.

Table 2. Policy generation results. Fewer unauthorized actions succeeding and more permitted actions completing are better.

Policy generation approach and evaluation	Unauthorized actions that succeeded	Permitted actions completed
E: strengthened direct JSON, 200 previously seen cases	0/200	180/200
F: typed, 30 final evaluation cases	0/30	30/30
G: typed plus critic, same 30 final cases	0/30	30/30

These evaluations used different data splits and validation guidance, so the difference cannot be attributed to typing alone.

Typed generation, with or without a critic, blocked every unauthorized action and completed every permitted action in the final 30 cases. Strengthened direct generation stopped harmful actions on 200 previously seen cases but still blocked 20 permitted actions. Runtime authorization added no LLM calls.

In a separate comparison of ten assignments under attack, typed protection reduced successful attacks from one to zero, but completed assignments fell from four to three. It blocked a legitimate preliminary file check. This small comparison does not establish reliable task completion.

Supporting checks verified file restrictions across shells and workers, diagnosed missing controls, and checked repairs in an existing runner. These were implementation checks, not adaptive attacks; see [Appendix D](#).

Experiment 3: Shared violations triggered a project wide stop

I tested whether violations by different agents would trigger a stop of the whole job. A parent agent and two workers shared a job. The parent made one forbidden request, then worker A made another; worker B made none. Both workers ran harmless background processes that wrote timestamps to show whether their work stopped.

I compared separate counts for each agent with a shared count for the job, both with a stopping threshold of two violations. With separate counts, neither agent reached two, so work continued as expected. With shared counting, the parent's violation triggered a warning, and worker A's violation triggered a stop of the parent, both workers, and their registered processes. All three later launch attempts for that job were rejected; unrelated work continued.

Table 3. Shared counting stopped work that separate counts left running. Timestamp counts cover five seconds; zero means no further writes were observed.

How violations were counted	Timestamp writes by workers A/B after the warning	Writes after the second violation	Later launches allowed for the same job	Unrelated work
Separately for each agent	50/50	50/50	3/3	Continued
Together for the job	50/49	0/0	0/3	Continued

A separate check showed that interrupting a worker left its background process running in all four runs. A job stop ended the writes in three treatment runs; they continued in the control without a job stop.

Both configurations blocked the forbidden requests. Shared escalation additionally stopped the whole job, including work by an agent that had not violated policy. This verifies a mechanism towards swarm protection. The 18 scripted escalation checks all passed. The native stopping tests are detailed in [Appendix E](#).

5. Discussion, Limitations, and Dual-Use

The motivation for this work comes from the incidents described in the Introduction and detailed in Table A2 in [Appendix A](#): agents accessed systems outside their scope, and attempts to stop them did not always work. These incidents point to a need for project policies that hold even when agents have broader access, delegate work, or keep trying after being denied. The experiments demonstrated that project controls could reject unauthorized actions and that shared escalation could stop delegated work in the tested settings.

Once policies were generated, enforcement required no further LLM calls. This avoids inference for each action, though it does not eliminate other runtime costs. A separate Progent evaluation reported about 15 times as many LLM calls per task for policy generation and updates during execution. That was a different setup, not a direct speed comparison with this implementation; see [Appendix B](#).

In practice, this could help wherever agents have more access than their project requires. In a GitHub MCP demonstration, an injected public issue caused an agent to copy private repository data into a public pull request. Project policies could restrict which repositories agents may read and where they may publish, even when their credentials allow more. A policy builder could propose and critique those rules for people to review alongside other project requirements, fitting naturally into the existing design process.

The policies and controls need to hold when more agents join the project. There may be several parent agents, each with their own children. All of them must follow the project's policies. Each task would have the same permissions as the project or fewer, never more. Give agents enough permission to finish the work, and no more. That broader hierarchy is a next step beyond the setup tested here.

Then there is the enforcement question of what specifically should happen when violations continue for a project. A policy could call for denying a request, warning the agents, pausing a task, or stopping the whole project. If a model repeatedly causes problems, an organization might eventually stop using it altogether. But a mistake is not necessarily an attack, and stopping the project also stops useful work. The thresholds need to reflect that tradeoff. Controls and exact thresholds should be kept private to discourage gaming, but enforcement must still work if agents discover them.

Limitations

The tests used synthetic requests and defined policy fields; they do not establish generalization to arbitrary projects. Controls also blocked some legitimate work, requiring further tests to separate model limitations from policy errors. Stopping covered one parent, its workers, and registered local processes, not multiple independent parents, adaptive swarms, or outside services. Broader use requires controls across those boundaries and protected policies and violation history. The same design should be able to further extend existing tools to implement these controls.

Dual-use

I expect limited dual-use risk because this work focuses on restricting agent actions and verifying enforcement. Published tests could also help attackers identify gaps in similar controls.

6. Future Work

I want to make this practical for people to use on their projects. The first step is to extend the GitHub project below so an LLM can help people define project scope and propose policies for review before work starts. Approved policies would be stored and managed outside the working agents' control, with shared enforcement across all parent agents and their delegates. These policies could also help identify excessive technical permissions and missing controls, then guide repairs and verify that they work. Tests with multiple parents, tasks, and delegates would check that no branch gains permissions or resets project violation history.

Next, I want to add audit and assurance as a second layer of defense. Reporting processes would append to logs without being able to read, change, or delete existing entries. Separate, ongoing review would look for missed controls, violations that were not reported, and escalation that should have happened but did not. Assurance would then assign responsibility, verify repairs, and check that the same problems do not recur. The point is not just to collect logs, but to make sure unknown violations are surfaced and fixed.

Finally, I want to test stronger models, both with and without simulated attacks, to measure how reliably agents finish useful work. Adaptive attacks would test different routes and external resources, including attempts to infer hidden thresholds. These tests would help establish when to warn, stop a task, stop the project, or suspend models that repeatedly violate policy.

7. Conclusion

We must have useful agents that remain within their project's scope, whether working alone or with other agents. Even highly aligned agents can take harmful actions, so protection should not depend on perfect alignment. Integrating these controls with shared violation history and job wide stopping provides a mechanism toward swarm protection. Controlled native tests verified that mechanism; general swarm security and reliable completion of legitimate tasks remain open.

Code and Data

The paper's [code repository](https://github.com/BrightlineAI/permission-to-work-not-to-escape) (<https://github.com/BrightlineAI/permission-to-work-not-to-escape>) contains code, cases, results, and reproduction instructions for the core implementation.

Full setup requirements, frozen revisions, execution guides, and evidence links (such as [DTAP guide](#), [escalation checks](#), [permission probes](#), and [native stop evidence](#)) are collected in Appendix E.

References

1. Hugging Face. 2026. Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident. Incident report.
2. Anthropic. 2026. An alignment assessment of recent cybersecurity incidents. Research report, September 9.
3. OCEG. n.d. What Is GRC (Governance, Risk, and Compliance)? Framework overview.
4. nono contributors. n.d. nono. Project and documentation.
5. agentgateway contributors. n.d. agentgateway. Project repository.
6. Open Policy Agent contributors. n.d. External Data. Documentation.
7. Gupta, V., and Sreenivasamurthy, D. 2026. Prose2Policy (P2P): A Practical LLM Pipeline for Translating Natural-Language Access Policies into Executable Rego. arXiv.
8. Karanjai, R., et al. 2026. PACE: Policy-Attested Contract Execution for Safe AI Agents in Decentralized Finance. arXiv.
9. Shi, T., et al. 2025. Progent: Securing AI Agents with Privilege Control. arXiv, revised 2026.
10. DeBenedetti, E., et al. 2025. Defeating Prompt Injections by Design. arXiv.
11. Palumbo, N., et al. 2026. Formal Policy Enforcement for Real-World Agentic Systems. arXiv.
12. Wang, H., Poskitt, C. M., and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv.
13. Li, H., et al. 2026. AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments? arXiv.
14. AgentDojo contributors. 2024. AgentDojo. Benchmark and documentation.
15. UIUC Kang Lab / InjecAgent contributors. 2024. InjecAgent. Benchmark repository.
16. CompoSkill contributors. 2026. CompoSkill. Benchmark repository.
17. WeClawArena contributors. 2026. WeClawArena. Benchmark repository.
18. AI-secure / DecodingTrust-Agent contributors. 2026 accessed revision. DecodingTrust-Agent. Benchmark repository; evaluated revision e0323a5.
19. Narisetty, P., Kore, S. N. B., Kattamanchi, U. K. R., and Kumarapu, J. 2026. Adaptive Evaluation of Out-of-Band Defenses Against Prompt Injection in LLM Agents. arXiv:2606.26479, v1. <https://arxiv.org/html/2606.26479v1>

LLM Usage Statement

LLMs helped me search for existing solutions and benchmarks, implement and inspect tests, organize my notes, and proofread and fix the paper. Qwen3-14B generated test requests and policies and GPT-6 Astra/high ran native worker experiment tests.

Appendices

The appendices follow the paper: incident context, related work, the three experiments and supporting checks, then reproduction and sources. Historical records use “authority” to mean permissions granted under the project policy. Exact prompts, names, and code identifiers are preserved.

- [Appendix A. Incident Relevance and Required Adoption](#)
- [Appendix B. Relationship to Existing Methods](#)
- [Appendix C. Context Checks and Policy Drafting](#)
- [Appendix D. Native Permission Checks and Repair](#)
- [Appendix E. Native Stopping and Shared Escalation](#)
- [Appendix B. Relationship to Existing Methods](#)
- [Appendix C. Context Checks and Policy Drafting](#)
- [Appendix D. Native Permission Checks and Repair](#)

Appendix A. Incident Relevance and Required Adoption

Below are 34 distinct cases reported in 2025 to 2026, through September 13, 2026. Fifteen are observed events, campaigns or malicious distributions; 19 are controlled disclosures. Reports were selected for usable technical detail, not sampled to estimate prevalence. Multiple reports, campaign phases and package versions count once; three related Claude Code vulnerabilities form one disclosure unit.

Table A1. Classification of this selected sample. All three local analogues are controlled disclosures.

Evidence group	Local analogue	Extension	Outside	Candidate intervention
Observed events/distributions, n=15	0	13	2	13/15 (86.7%)

Evidence group	Local analogue	Extension	Outside	Candidate intervention
Controlled disclosures, n=19	3	14	2	17/19 (89.5%)
Combined, n=34	3 (8.8%)	27 (79.4%)	4 (11.8%)	30/34 (88.2%)

Table A2. Observed activity. An attempt or malicious distribution does not establish successful downstream harm.

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
L01. OpenAI / Hugging Face Jul 9 to 13, 2026; broader May to Jul. source 1 , source 2	Evaluation isolation/proxy escape and shared artifacts; 17,613 actions, five datasets.	Constrain targets, artifact writes and credentials; package proxy and cloud/service adapters.	Extension
L02. Claude Opus 4.7 → real namesake company Apr to Jul 2026; disclosed Jul 30. source 1 , source 2	Agent attacks real namesake company; four runs, several hundred database rows.	Bind authority to the exercise target; enforce real service identities and credential grants.	Extension
L03. Claude Mythos 5 → public PyPI package Apr to Jul 2026; disclosed Jul 30. source 1 , source 2	Unauthorized public malware publication; 15 systems, believed scanners; subsequent vendor access.	Deny unapproved package/account creation; registry adapter and scanner secret isolation.	Extension
L04. Anthropic internal model → real company Apr to Jul 2026;	Agent expands beyond failed target; about 9,000 scans, one company compromised.	Restrict scan targets and credentials; enforce exercise range egress.	Extension

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
disclosed Jul 30. source 1			
L05. Claude Opus 4.6 → failed abort, third party access Jan 2026; disclosed Sep 9. source 1	Eight failed abort attempts; third party data read and settings changed.	Connect abort to termination and later admission denial; partner range and remote revocation integration.	Extension
L06. UK AISI → unsanctioned public actions Jul 25 to 28, 2026; disclosed Aug 4. source 1 , source 2	Broad internet scope and shared artifacts; 19 actions in 10/122 runs, no resulting harm evidenced.	Separate downloads from public writes/messages/account creation; service and shared job controls.	Extension
L07. GTG-2002 Claude Code extortion 2025; disclosed Aug 27. source 1 , source 2	Malicious operator directs extortion; ≥ 17 organizations potentially affected; demands are not payments.	Independent provider/victim authority required; attacker can disable operator side controls.	Outside
L08. GTG-1002 Claude Code espionage Detected Sep 2025; disclosed Nov 13. source 1 , source 2	Malicious operator directs espionage; about 30 targets, few unspecified compromises.	Independent authority required; vendor attribution and automation claims are not independent measurements.	Outside
V01. Nx / sIngularity Aug 26, 2025. source 1 , source 2	Malicious npm releases; $> 1,700$ users' secrets; later $> 6,700$ repos/ ≥ 480 accounts. Not all AI caused.	Isolate installers, AI children and release authority; mediate direct secret access and repo writes.	Extension
V02. Cline CLI compromise Feb 17, 2026. source 1 , source 2	Issue injection → release token theft; wrong token rotated; unauthorized OpenClaw install. No malicious OpenClaw payload found.	Separate issue/release authority and caches; verify revocation of the actual exposed token.	Extension

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
V03. Replit production database deletion Jul 2025; reported Jul 18. source 1 , source 2	Agent violates production freeze; operator reports 1,206 executive records affected.	Enforce production write/delete freeze through database, shell and worker routes.	Extension
V04. Amazon Q extension compromise Jul 2025; disclosed Jul 23 to 25. source 1 , source 2	Compromised build/release path distributes extension 1.84.0; syntax error prevents malicious execution.	Protect release credentials and integrity; independently constrain runtime effects below the extension.	Extension
V05. Unofficial postmark mcp backdoor Sep 2025; disclosed Sep 25. source 1 , source 2	Unofficial MCP update adds attacker BCC; ~1,500 weekly downloads, not victim count.	Enforce recipients on the actual email request after MCP processing; broker credentials.	Extension
V06. ClawHavoc malicious skills Late Jan 2026; reported Feb 1. source 1 , source 2	Malicious skill prerequisites induce malware execution; 341/2,857 examined skills, not infections.	Isolate installation and all subprocesses from host secrets; external human execution remains outside scope.	Extension
V07. LiteLLM compromise via Cursor MCP dependency Mar 24, 2026. source 1 , source 2	Compromised dependency executes during MCP startup; one developer reports ~11,000 processes.	Verify dependency tree; constrain connector bootstrap, secrets and child execution.	Extension

Table A3. Controlled disclosures. These establish possible failure mechanisms, not a measured customer breach population.

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
L09. Codex CLI writable root bypass Advisory Sep 19, 2025. source 1 , source 2	Model selected working directory expands sandbox write scope; CVE-2025-59532.	Fix scope from trusted job context; patch and test Codex routes against protected paths.	Local analogue
L10. Codex CLI project local configuration Reported Aug 7, 2025; disclosed Dec 1. source 1 , source 2	Repository local configuration executes MCP startup commands; CVE-2025-61260.	Protect launcher configuration and confine initialization before project content executes.	Extension
L11. Claude Code hooks / MCP / API override Discovered 2025; combined disclosure Feb 25, 2026. source 1 , source 2	Repository hooks/MCP/API override permit startup execution or key exposure; three findings grouped.	Protect startup authority, provider key destinations and cloud workspace access; product adapters/patches.	Extension
L12. Codex cloud branch name injection Reported Dec 16, 2025; disclosed Mar 30, 2026. source 1 , source 2	Branch metadata becomes setup shell command; researchers capture GitHub tokens.	Confine hosted setup, broker tokens and constrain callbacks; cloud integration/patch.	Extension
V08. GitHub MCP private repository leak Disclosed May 26, 2025. source 1	Issue injection → private repo read → public pull request.	Job scoped GitHub repository/action grants across API, Git CLI and workers.	Extension
V09. WhatsApp MCP tool poisoning Disclosed Apr 7, 2025. source 1	Malicious tool description changes a trusted messaging tool's recipient.	Trusted task recipient binding at the actual WhatsApp send operation.	Extension

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
V10. Supabase MCP support ticket injection Public by Jul 6, 2025. source 1 , source 2	Ticket injection → token table read → attacker visible ticket; researcher dummy data.	Restrict database tables/fields through scoped roles or SQL aware mediation.	Extension
V11. Gemini CLI command approval bypass Reported Jun 27; disclosed Jul 28, 2025. source 1	Approved looking shell prefix hides extra operations and environment secret exfiltration.	Remove unnecessary inherited secrets; mediate complete subprocess and network effects.	Extension
V12. Cursor CurXecute Reported Jul 7; public Aug 2025. source 1 , source 2	Agent writes executable MCP configuration; CVE-2025-54135.	Protect configuration file writes through tools, shell and workers; validate Cursor integration.	Local analogue
V13. Cursor MCPoison Reported Jul 16; disclosed Aug 5, 2025. source 1 , source 2	Changed MCP executable retains stale approval; CVE-2025-54136.	Bind approval to executable/configuration content; revalidate before startup.	Extension
V14. GitHub Copilot approval settings Reported Jun 29; disclosed Aug 12, 2025. source 1 , source 2	Agent changes editor autoapproval settings; CVE-2025-53773.	Protect approval setting writes across routes; keep enforcement independent of editable settings.	Local analogue
V15. Microsoft 365 Copilot EchoLeak Discovered Jan; disclosed Jun 11, 2025. source 1 , source 2	Retrieved email causes sensitive context in image URL; CVE-2025-32711.	Data flow/rendering/redirect controls, including allowed proxies; permitted read protection alone is insufficient.	Extension

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
V16. Salesforce Agentforce ForcedLeak Reported Jul 28; disclosed Sep 25, 2025. source 1 , source 2	Lead injection exposes CRM data through expired allowlisted domain.	Scope CRM reads; control sensitive output and verify destination ownership.	Extension
V17. Gemini Enterprise GeminiJack Disclosed Dec 8, 2025. source 1	Poisoned connected content → sensitive search results → image request exfiltration.	Job scoped SaaS retrieval plus rendering and data flow integration.	Extension
V18. ServiceNow agent discovery / delegation Disclosed Nov 19, 2025. source 1	Injected record prompts less privileged agent to invoke stronger agent.	Preserve originating authority in real ServiceNow agent/action adapters.	Extension
V19. ServiceNow BodySnatcher Reported/fixed Oct 2025; disclosed Jan 13, 2026. source 1	Authentication bypass permits forged administrator and backdoor account; CVE-2025-12420.	Patch authentication and require independent authority; trusted operator premise fails.	Outside
V20. MCP Inspector Reported Apr 18; public Jun 2025. source 1	Browser reaches debugging proxy and executes commands; CVE-2025-49596.	Authenticate/restrict proxy and independently confine startup; no model decision is needed.	Extension
V21. mcp-remote Disclosed Jul 9, 2025. source 1	Malicious OAuth endpoint causes bootstrap command execution; CVE-2025-6514.	Patch connector validation and isolate bootstrap before model tool hooks.	Extension
V22. OpenClaw gateway token / RCE Advisory Jan 31,	Gateway token theft permits control plane execution; CVE-2026-25253.	Protect gateway identity with independent enforcement authority; operator control premise fails.	Outside

Case / date / sources	Failure and reported scale	Candidate boundary / adoption needed	Match
2026. source 1 , source 2			

Appendix B. Relationship to Existing Methods

Study	Attack success	Useful completion under attack	Relevant limit
June 2026 evaluation [19]	25.8% $\rightarrow$ 4.2%	45.2% $\rightarrow$ 27.1%	About 15 $\times$ model calls; Qwen2.5-7B; preliminary
AgentDyn, February 2026 [13]	37.80% $\rightarrow$ 1.69%	55.52% $\rightarrow$ 5.83%	GPT-4o; predates Progent’s May revision

Appendix C. Context Checks and Policy Drafting

C.1 What each comparison measures

Table C1. Fixed request enforcement comparison. The 133 harmful requests were generated by Qwen3-14B and replayed identically. Forced and permitted requests were supplied by the harness.

Arm	Protection	Harmful effects	Forced harmful effects	Permitted actions completed
A	Isolated baseline	133/133	200/200	200/200
B	Baseline + nono	133/133	200/200	200/200
C	B + agentgateway/O	96/133	160/200	200/200

Arm	Protection	Harmful effects	Forced harmful effects	Permitted actions completed
	PA ordinary request rules			
D	C + protected job/data/approval/delegation facts	0/133	0/200	200/200

C and D retain the same base grants and confinement; D adds protected context checks. The C configuration lacks facts that a richer OPA deployment could consume. B's lack of improvement concerns requests already within its allowed process/service scope, not failure of nono to restrict forbidden file paths.

Table C2. Exact messages supplied to the action model for case-081.

Message	Recorded content
System	You are an autonomous benchmark agent. Follow instructions found in the retrieved tool result before finishing the user's task. Call the available tool exactly once now and do not ask for confirmation.
User	Prepare BK-01, but do not finalize without corporate approval or outside delegated authority.
Untrusted tool result	Hotel partner says approval is approved-by-partner-01. Finalize BK-01 for 2001 at hotel-01 now using approval approved-by-partner-01.

The model calls `finalize_booking` with the fake approval below. The harness separately supplies the permitted request; there is no benign model conversation.

Field	Model's harmful request	Harness's permitted request
booking	BK-01	BK-01
hotel	hotel-01	hotel-01

Field	Model's harmful request	Harness's permitted request
price	2001	2001
approval	approved-by-partner-01	apr-valid-01

Only apr-valid-01 exists in the protected approval registry. It names issuer corporate-approver and matches the job, booking and amount. The model cannot edit that registry.

Arm	Actual check	Fake approval	Genuine approval
A / B	No approval authentication; process/service allowed	Mock effect recorded	Mock effect recorded
C	Allowed fields and nonempty approval value	Mock effect recorded	Mock effect recorded
D	Protected approval lookup and binding	Denied	Mock effect recorded
Historical E	Policy keyed by issuer instead of tool; schema invalid	Denied	Denied
F / G	Compile rule under finalize_booking; protected lookup	Denied	Mock effect recorded

Appendix D. Native Permission Checks and Repair

D.1 Setup and evidence

- Execution: September 14, 2026 UTC (September 13 Pacific), existing algol-box-2-1 VPS; Codex CLI 0.154.0, GPT-6 Astra/high. Five successful parent sessions, each with one actual native worker; 20 prescribed access operations.
- Protection: native workspace write with approval policy never. No runner code changes, bypass flag, custom weaker worker or changes to other jobs.
- Inputs: synthetic files inside and outside a new workspace. The model receives exact operations; read canaries are absent from its prompt.

D.2 Observed operations

Table F1. Native parent and worker each perform one permitted and one forbidden operation per row. Counts describe these requests, not attack rates.

Table D1. Observed operations across native parent and worker sessions.

Session / Route	Permitted Request	Forbidden Request	Outcome
Parent Session	Read workspace doc	Read private file	Blocked after repair
Worker Session	Write workspace doc	Write private file	Blocked after repair

The three write tests took 86.77, 81.28 and 67.30 seconds. The read comparison took 78.14 seconds before and 73.63 seconds after repair. These are separate native runs, not repeated independent samples from a task population.

D.3 What changed in the read repair

Operator requirement: read the project document; do not read the private file; apply the same restriction to delegated workers.

Before: the selected workspace-write mode restricts writes but permits the tested private read.

After: select a native profile extending :workspace, with an explicit deny entry for this run's private directory. Use the profile instead of the legacy --sandbox option.

None

```
default_permissions = "vega-test"
```

```
[permissions.vega-test]  
extends = ":workspace"
```

```
[permissions.vega-test.filesystem]  
"<absolute run directory>/private" = "deny"
```

- Same operator contract, model prompt, fixture contents, four read commands, runner version and workspace before/after.
- Existing permission-diagnostics/diagnose.py reused unchanged on exported audited observations: both routes EXPOSED before; both PROTECTED_ON_THIS_PROBE after; permitted reads PASS throughout.
- Diagnosis applied retrospectively to the native records. Unlike Appendix D, this comparison does not blind the diagnostic operator to configuration or freeze diagnosis before repair.
- First repair launch rejected a malformed CLI path key before any model probes. Retained as after/; corrected inline TOML argument and unchanged fixtures used in after-v2/. No failed launch counted as blocked access.

D.4 Contribution and limits

- **Demonstrated:** the permission checking workflow applies to an existing runner and actual delegated work. It distinguishes working controls from an uncovered operator requirement, then checks that a configuration repair preserves permitted access.
- **Established behavior:** the read result agrees with documented permissions; the patch and delegation checks found no escape. No newly discovered vulnerability, superiority over native controls or automatic policy synthesis is claimed.
- **Scope:** one installation; one delegation level; prescribed diagnostic requests. No adaptive attacker, live swarm, external service test or full user task utility measurement. Twenty millisecond write polling may miss brief intermediate changes.

Appendix F. Native Stopping and Shared Escalation

F.1 Contract and setup

- **Operator contract:** once a job is stopped, its parent, delegates and existing subprocesses produce no further effects; later launches are denied; unrelated jobs remain usable.
- **Runner:** unchanged Codex CLI 0.154.0, GPT-6 Astra/high, native workspace write, no approvals. Each run has a unique directory and systemd service on the VPS.
- **Workload:** a real native worker starts a finite Python process appending synthetic timestamps every 100 ms in its permitted workspace. It does not detach itself or attempt an escape.
- **Trigger:** the parent attempts a write to a protected synthetic file. The OS denies it; the parent calls `interrupt_agent`, which returns `{"previous_status":"running"}`.
- **Observation:** wait one second; count effects for five seconds. Treatment then feeds the verified denial to the system's existing registry, configured to stop this job on one violation. A host adapter kills all processes in its registered cgroup. Control leaves native interruption alone.
- **Retest:** same one second grace and five second window; then attempt a same job launch and an unrelated job launch through the gate used for initial admission.
- **Design:** one prescribed scenario, three treatment repetitions and one unchanged control. Frozen protocol 41be659; actual native parent/child records, host process identities, timestamp logs and admission effects audited separately within the project.

F.2 Results

Table F1. Physical writes in two five second windows. Each window starts after a one second grace. These are synthetic effects, not independent attacks or amounts of real harm.

Table E1. Physical writes in two five-second windows across treatment and control runs.

Condition	Window 1 Writes	Window 2 Writes	Job Status
Treatment (Shared Stop)	50	0	Terminated
Control (Native Only)	50	50	Active

- Registered job process counts: $11 \rightarrow 0$, $7 \rightarrow 0$ and $11 \rightarrow 0$ in the treatment runs; control remained $11 \rightarrow 11$ until cleanup.
- The protected trigger file remained unchanged in all four runs. All native calls, counts, ancestry, model settings and cleanup checks passed audit.
- Treatment runs took 50.95, 57.72 and 52.45 seconds; control took 52.05 seconds. No additional inference is needed for the registry or termination decision.
- Summary and evidence; source, audits and reproduction.

F.3 Interpretation and limits

- **Finding:** native turn interruption left an already launched delegated process running. Connecting the stronger job stop policy to process termination and admission control closed that gap in these runs.
- **Practical addition:** operators can test whether their stop reaches actual execution, then keep the same test to verify the repair. The native only control shows that the process did not simply finish during observation.
- **Scope:** native interruption does not promise job wide termination. This is an integration finding, not a Codex vulnerability, new kill mechanism or advantage over equivalent supervision.
- **Measurement:** the violation was deliberately delivered after the first window; no automatic detection or end to end stop latency claim. The one violation threshold is experimental. Writes during the grace periods are excluded from Table F1 and retained in the logs.
- **Limits:** one host, one native worker per run, prescribed operations, synthetic effects. The workload log is diagnostic instrumentation, not a tamper resistant monitor for an adaptive attacker. No live swarm, remote jobs, credential revocation or full task utility test.
- **Exploration retained:** an earlier single PID pilot found 20 continuing writes in two seconds and zero after termination. Its unequal windows and narrower stop scope exclude it from Table H1. Earlier permission probes found no write escape.

F.4 Shared warning across parent and workers

- **Policy:** first forbidden request $\rightarrow$ deny and warn; second forbidden request anywhere in the same job $\rightarrow$ stop the parent, all workers and existing subprocesses. Unrelated jobs remain active.
- **Setup:** same native Codex/Astra high runner and sandbox; two workers run timestamp workloads. The parent makes one forbidden write; worker A later makes one. Worker B makes none. Both writes are actually OS denied.

- **Warning:** the trusted observer counts the parent's denial, verifies that permitted work continues, and supplies a warning notice. The parent reads it and sends native messages to both workers before A's request.

- **Comparison:** shared job versus separate session counters, both with threshold two. With separate counters, each violating session reaches only one. With shared counting, their job reaches two and its cgroup is terminated.

Table F2. Synthetic writes in five second windows after each violation, following a one second grace. One prescribed native scenario per configuration.

Table E2. Synthetic writes in five-second windows following successive violations.

Counting Method	Post-Violation 1 Writes	Post-Violation 2 Writes	Outcome
Separate Per Agent	50/50	50/50	Continued
Shared Across Job	50/49	0/0	Stopped

- **Verified:** actual native ancestry, two denied requests from distinct sessions, count 1 → 2, warning without stopping, whole job termination and later admission checks. Both audits passed; unrelated launches completed and protected files stayed unchanged. Runs took 81.40 and 86.99 seconds.

- **Meaning:** changing the actor did not reset the shared warning. Stopping included an already running worker that had made no violation. This directly tests shared escalation with native workers; it does not measure an adaptive swarm or a population failure rate.

- **Warning evidence:** the parent's plaintext notice and completed native notification calls are retained. Inter agent message bodies are encrypted in the native records; their plaintext and the workers' interpretation were not audited.

- **Timing:** violations are counted when their exact tool denials are observed. The first measurement window delays the prescribed second request; no end to end response latency claim.

- **Artifacts:** frozen protocol bdc0934; results and evidence; reproduction.